



Security Audit

VIA Labs (Stellar)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	72
Conclusion	73
Our Methodology	74
Disclaimers	76
About Hashlock	77

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Via Labs team partnered with Hashlock to conduct a security audit of their programs. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

VIA Labs is a cross-chain infrastructure protocol that specializes in bridging and messaging solutions. Their flagship product focuses on USDC bridging and Proto-USD implementation across multiple blockchains. The protocol enables secure and efficient cross-chain communication while maintaining asset stability through their unique gas reimbursement mechanism.

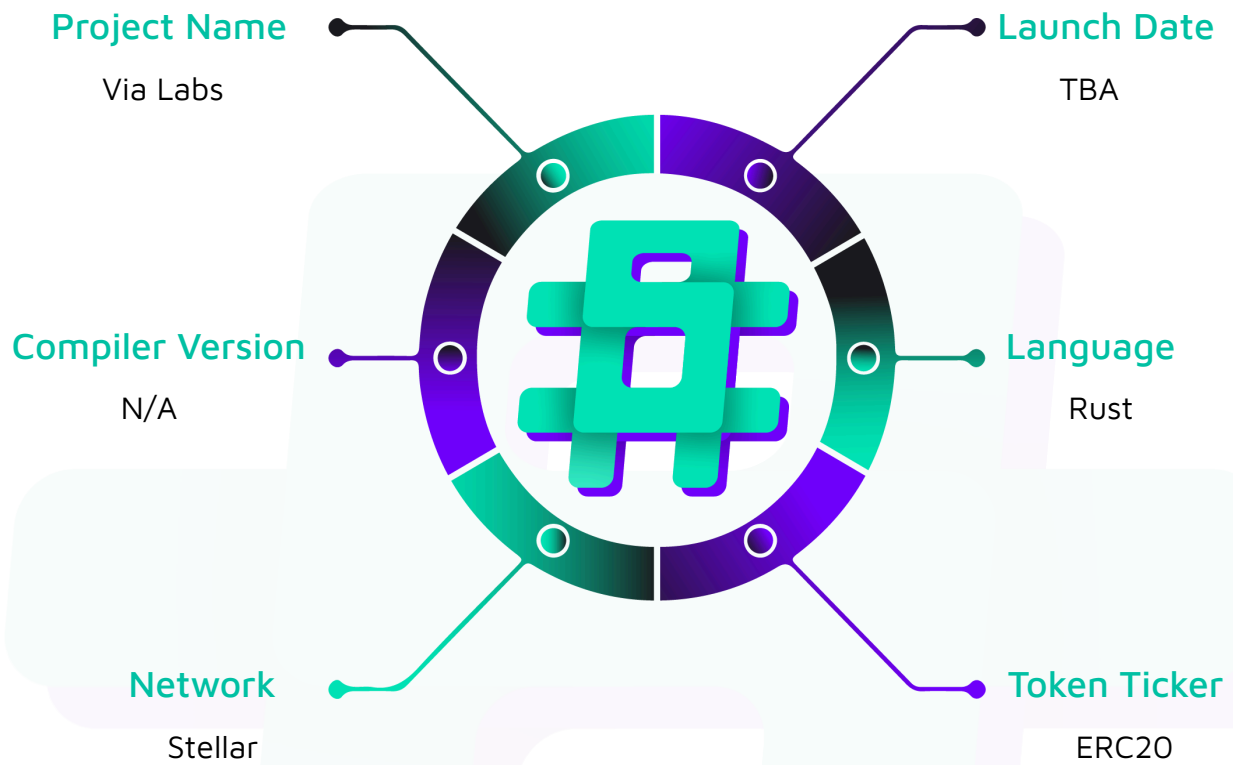
Project Name: VIA Labs

Project Type: Cross-Chain Infrastructure

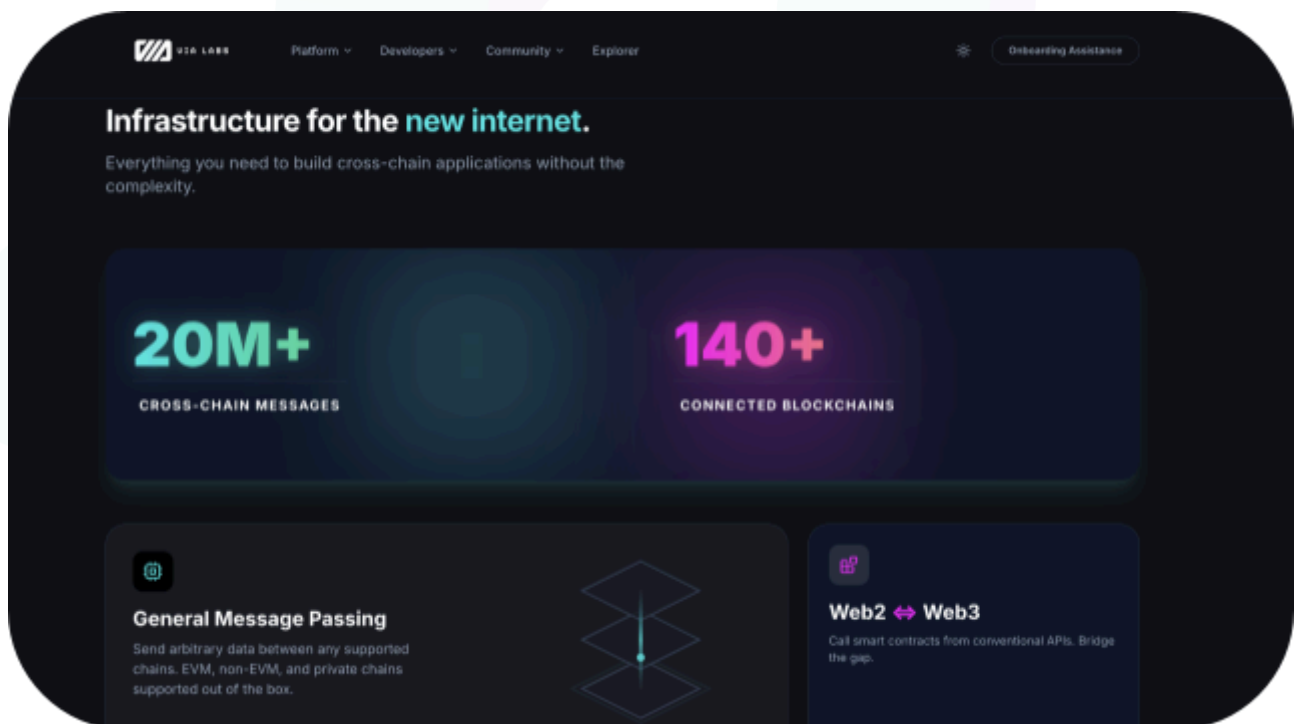
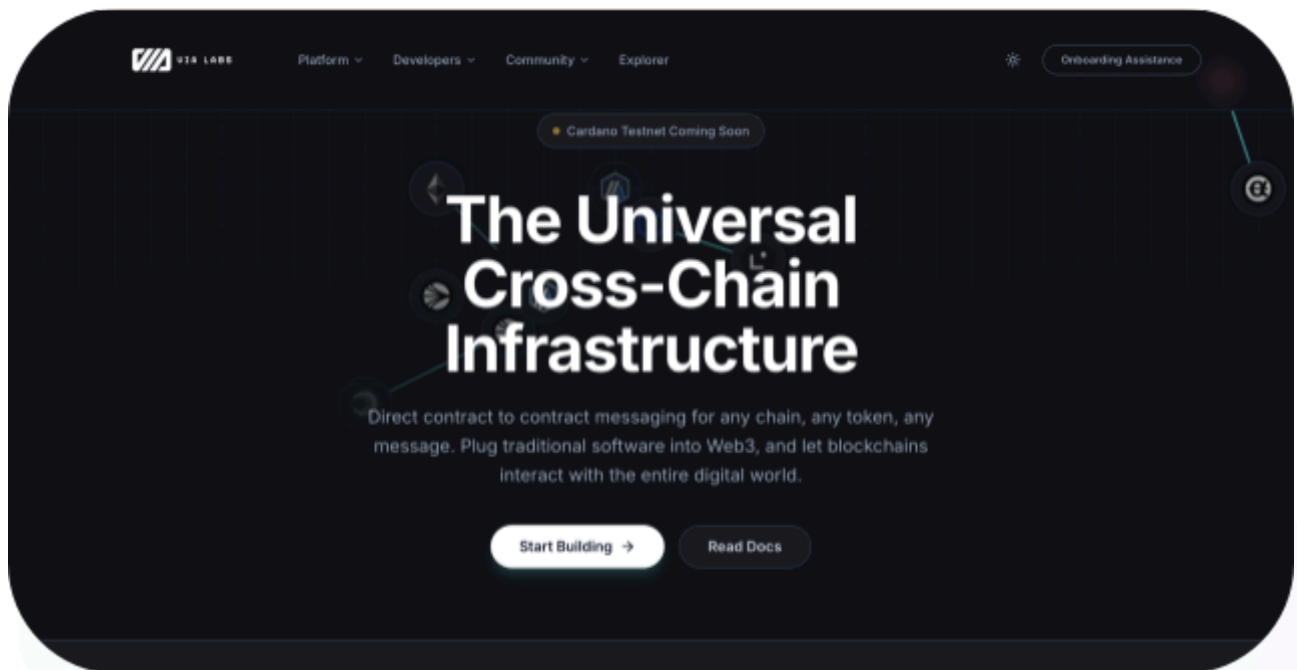
Website: <https://vialabs.tech/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the Rust code within the Via Labs project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Via Labs Programs
Platform	Soroban / Rust
Audit Date	May, 2026
Program 1	contracts/stellar/client-test-v4/src/lib.rs
Program 2	contracts/stellar/fee-handler/src/errors.rs
Program 3	contracts/stellar/fee-handler/src/events.rs
Program 4	contracts/stellar/fee-handler/src/interface.rs
Program 5	contracts/stellar/fee-handler/src/lib.rs
Program 6	contracts/stellar/fee-handler/src/storage.rs
Program 7	contracts/stellar/gas-handler/src/errors.rs
Program 8	contracts/stellar/gas-handler/src/events.rs
Program 9	contracts/stellar/gas-handler/src/interface.rs
Program 10	contracts/stellar/gas-handler/src/lib.rs
Program 11	contracts/stellar/gas-handler/src/storage.rs
Program 12	contracts/stellar/message-client/src/macros/src/lib.rs
Program 13	contracts/stellar/message-client/src/abi.rs
Program 14	contracts/stellar/message-client/src/lib.rs
Program 15	contracts/stellar/message-gateway-v4/src/errors.rs
Program 16	contracts/stellar/message-gateway-v4/src/events.rs

Program 17	contracts/stellar/message-gateway-v4/src/interface.rs
Program 18	contracts/stellar/message-gateway-v4/src/lib.rs
Program 19	contracts/stellar/message-gateway-v4/src/storage.rs
Audited GitHub Commit Hash	c8f23dd8e4afff01417ca9cf437d8d5cec730f92
Fix Review GitHub Commit Hash	f8a548d103ef9f4dd6d4adf778a93c0744ddc146

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. The protocol uses Rust-based Soroban smart contracts compiled to WASM, alongside custom interfaces, storage abstractions, and handler-based routing components for cross-chain messaging and fee management.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities we have identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

3 Medium severity vulnerabilities

21 Low severity vulnerabilities

6 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Code Quality

This audit scope involves the smart contracts of the Via Labs project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The codebase is well documented and follows clear Rust documentation and Soroban development conventions. Comments are generally aligned with the implemented functionality, and the modular contract structure improves readability and maintainability throughout the system.

Audit Resources

We were given the Via Labs project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the Soroban smart contract infrastructure uses established Rust and Stellar ecosystem libraries alongside custom modular components. Additionally, the protocol interacts with external contracts through cross-contract invocation patterns within the Soroban environment.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] message-gateway-v4/src/lib.rs#process - Replay attacks using messages to other gateways

Description

The Stellar gateway signs and verifies the claimed `destination_chain_id`, but it never checks that value against its own actual `chain_id`, stored at construction in `contracts/stellar/message-gateway-v4/src/lib.rs:101-118`.

Vulnerability Details

Function `process()` never compares `request.destination_chain_id` to the stored gateway `chain_id` in `contracts/stellar/message-gateway-v4/src/lib.rs:263-310`. The field `destination_chain_id` is only included in the signed payload in `contracts/stellar/message-gateway-v4/src/lib.rs:705-723`.

VIA's autogenerated recipient code validates the message source and sender route, but it does not validate the destination chain. That check belongs at the gateway, which must reject messages not addressed to its own configured chain ID.

A relayer can submit a message carrying `destination_chain_id = X` to a gateway actually running on chain `Y`, and the gateway will accept it as long as the signatures are valid for the submitted payload. The gateway is only verifying that the payload was signed, not that it was intended for exactly that gateway.

Proof of Concept

```
#[test]
#[should_panic]
fn test_process_rejects_destination_chain_mismatch() {
    let fixture = TestFixture::new();
    fixture.enable_system();
}
```

```
fixture.setup_handlers();
fixture.setup_signers_with_keypairs(1, 1);
fixture.setup_relayers(true);
let client = fixture.gateway_client();
let mut request = fixture.create_process_request(1, &fixture.client_contract, 0);
request.destination_chain_id = DESTINATION_CHAIN_ID + 1;
request.signatures = fixture.sign_process_request(&request, &[0, 1]);
client.process(&request);
}
```

Impact

The impact depends on the recipient contract. It can range from confused delivery and invalid calls to targeted replay attacks and potential funds loss. Even if the lost funds are difficult to move in a controlled direction, extreme griefing is possible.

Recommendation

The gateway must validate payloads submitted by relayers and reject messages aimed at the wrong gateway.

Status

Resolved

[H-02] message-gateway-v4/src/storage.rs#DataKey - Gateway configuration DoS due to shared instance storage

Description

An attacker can exhaust the gateway's shared instance storage with bogus per-address namespaces created through public per-contract configuration setters.

Vulnerability Details

Tenant-specific namespaces are stored in gateway instance storage under address-keyed DataKeys in:

- contracts/stellar/message-gateway-v4/src/storage.rs:73-78
- contracts/stellar/message-gateway-v4/src/storage.rs:87-92

Both public setters are reachable without permission:

- contracts/stellar/message-gateway-v4/src/storage.rs:294-320
- contracts/stellar/message-gateway-v4/src/storage.rs:322-345

The gateway does not require prior registration and does not validate that the supplied namespace address is a contract.

An attacker can control arbitrary addresses and repeatedly call `set_contract_relayers` or `set_project_signers` for fresh addresses until the shared instance entry is saturated.

The attack is amplified because there is no registration or whitelist gate, per-namespace state is stored in the same shared instance entry as all gateway state, old membership keys are only flipped to `false` instead of deleted, and input vectors are not deduplicated.

Proof of Concept

```
#[test]
#[should_panic(expected = "Error(Budget, ExceededLimit)")]
fn test_bloat_eventually_blocks_new_relayer_namespace_configuration() {
    let (env, gateway) = new_authorized_fixture();
    let client = MessageGatewayV4Client::new(&env, &gateway);
    for _ in 0..200 {
        let namespace = Address::generate(&env);
        let relayers = create_addresses(&env, 64);
```

```
    client.set_contract_relayers(&namespace, &relayers);  
}  
}
```

Impact

This can block onboarding new clients, adding or rotating per-client relayers, onboarding new relayer namespaces for additional clients, and other future per-client configuration that accumulates in shared instance state. Functions `send()` and `process()` are not directly blocked by this DoS.

Recommendation

Move tenant-specific per-contract configuration from instance storage to persistent storage. Alternatively, move project-specific signer and relayer configuration out of the shared gateway contract entirely and into client-owned or per-client config contracts. Additionally, implement a whitelist or registration step with a security deposit before namespaces can be created, optimize the storage layout, and require sorted and deduplicated input vectors.

Status

Resolved

Medium

[M-01] message-gateway-v4/src/lib.rs#encode_via_message - Ambiguous message format allows signed-message malleability

Description

The gateway constructs the signed message hash by concatenating multiple fields directly, including variable-length fields such as `sender`, `on_chain_data`, and `off_chain_data`, without length prefixes or canonical framing.

Vulnerability Details

The affected logic is located in `contracts/stellar/message-gateway-v4/src/lib.rs:705-742`. The resulting byte stream is hashed and signature-checked in `contracts/stellar/message-gateway-v4/src/lib.rs:313-345`.

Different logical messages can map to the same byte stream and therefore the same signed hash.

The same ambiguity exists in the TypeScript signing path. The Stellar driver reconstructs the message hash off-chain in `src/drivers/StellarV4.ts:1073-1114` using `encodeViaMessage()` from `src/utils/stellar-encoding.ts:57-85`, which applies the same non-canonical concatenation logic.

An attacker would need a legitimately signed message, an alternative split of the same byte stream across `sender`, `on_chain_data`, and `off_chain_data`, and a second interpretation that is syntactically accepted and semantically useful to the recipient contract.

Impact

If two different message layouts produce the same encoded bytes, a signature set gathered for one message may also validate another. This can cause the gateway to call `message_process_from_gateway` with message contents that differ from what validators intended to approve.

Depending on how recipient contracts use `sender`, `on_chain_data`, and `off_chain_data`, this can lead to wrong actions being executed, altered sender context being trusted,



invalid payload data being accepted, loss of funds, invalid state transitions, or failure of core bridge trust guarantees.

Recommendation

Replace manual concatenation with canonical serialization of the full signed message, such as XDR, or include explicit length prefixes for every dynamic field before hashing and signature verification.

Status

Resolved

[M-02] fee-handler/src/lib.rs#__constructor - Collected fees become unrecoverable when `fee_recipient` is unset

Description

The fee handler starts in an unsafe default state because `__constructor` initializes owner, admin, and default fee amount, but does not store `FeeRecipient`.

Vulnerability Details

The constructor behavior is located in `contracts/stellar/fee-handler/src/lib.rs:132-141`.

If fee-bearing traffic begins before the owner calls `set_fee_recipient`, collected fees are redirected to the fee-handler contract itself and remain trapped under the current ABI.

Function `get_fee_recipient` falls back to `env.current_contract_address()` when `FeeRecipient` is unset in `contracts/stellar/fee-handler/src/lib.rs:84-89`.

Function `process_fee` loads that recipient and uses it for both fee paths in `contracts/stellar/fee-handler/src/lib.rs:241-292`.

The native source-fee path transfers directly to that resolved recipient in `contracts/stellar/fee-handler/src/lib.rs:249-263`, and the token-fee path does the same in `contracts/stellar/fee-handler/src/lib.rs:276-289`.

The public fee-handler interface exposes no owner-gated withdrawal or recovery path in `contracts/stellar/fee-handler/src/interface.rs:5-203`.

Impact

If operators configure the gateway to use the fee handler before setting the fee recipient, normal fee-paying traffic sends fees to the fee-handler contract itself. Future fees can be redirected after setting the correct recipient, but previously collected fees remain stuck because the reviewed ABI has no recovery path.

Recommendation

Initialize `FeeRecipient` during construction and fail closed if it is ever unset. Add owner-gated recovery functions for configured fee tokens and native source-fee balances so operators can sweep historical fees trapped before the fix.

Status

Resolved



Low

[L-01] message-client/src/lib.rs#set_message_gateway - In-place gateway rotation is unavailable for already-bound client contracts

Description

The Stellar message-client helper supports an initial gateway bind, but not an in-place rebinding through the same interface once a gateway is already set.

Vulnerability Details

The `set_message_gateway` in `contracts/stellar/message-client/src/lib.rs:224-234` behaves differently depending on whether an admin is present. The first call uses `set_admin` directly, while later calls switch into `transfer_admin_role` followed immediately by `accept_admin_transfer`.

The second branch does not match the call context provided by the helper itself. The owner-driven call begins with `enforce_owner_auth` in `contracts/stellar/message-client/src/lib.rs:225`, then enters `transfer_admin_role` and `accept_admin_transfer` in `contracts/stellar/message-client/src/lib.rs:227-231`. Those helpers require authorization from the current admin and then the pending admin, so the normal owner-driven flow cannot complete both steps.

Impact

A client that is already bound cannot simply be pointed at a new gateway by its message owner through this interface. Gateway rotation for upgrades, migrations, bug fixes, or key compromises may require redeployment or non-standard recovery.

Recommendation

Choose one lifecycle explicitly. If Stellar clients are meant to be permanently bound to their first gateway, remove the rebinding branch and fail with a clear error on any second `set_message_gateway` call. If gateway rotation should be supported, redesign it as a genuine two-step procedure where the new gateway completes acceptance from its own authorization context.

Status

Resolved

[L-02] `message-client/src/lib.rs#set_message_owner` - Delayed owner initialization allows takeover

Description

The Stellar `message-client` framework expects integrators to initialize the message owner during construction, but `#[default_impl]` only auto-generates helper methods and does not enforce any constructor shape.

Vulnerability Details

The macro behavior is located in `contracts/stellar/message-client/src/macros/src/lib.rs:44-146`.

If a project deploys the contract first and only later calls `set_message_owner`, the contract remains claimable by the first caller.

`set_message_owner` is a thin wrapper around `set_owner` in `contracts/stellar/message-client/src/lib.rs:210-212`, so the framework relies on integration convention rather than compile-time enforcement.

A user of `#[default_impl]` can forget to call `set_message_owner` in the constructor example shown in `contracts/stellar/message-client/src/lib.rs:20-22`, or can defer owner initialization operationally.

Proof of Concept

```
let Some(constructor) = input.items.iter_mut().find_map(|item| match item {
    ImplItem::Fn(function) if function.sig.ident == "__constructor" =>
    Some(function),
    _ => None,
}) else {
    return syn::Error::new(
        Span::call_site(),
        "#[default_impl] requires __constructor(env: Env, owner: Address, ...)",
    );
}
```

```

    )
    .into_compile_error()
    .into();
};
if !is_exact_type(&constructor.sig.inputs[0], "Env")
    || !is_exact_type(&constructor.sig.inputs[1], "Address")
{
    return syn::Error::new_spanned(
        &constructor.sig,
        "#[default_impl] requires __constructor(env: Env, owner: Address, ...)",
    )
    .into_compile_error()
    .into();
}
constructor.sig.ident = syn::Ident::new("__message_client_constructor",
Span::call_site());
pub fn __constructor(env: Env, owner: Address, ...) {
    message_client::MessageClientBase::set_message_owner(&env, owner.clone());
    Self::__message_client_constructor(env, owner, ...)
}

```

Impact

A front-runner can permanently seize the intended owner role if the contract is deployed in an unsafe ownerless state. In a typical integration, the attacker could control functions such as `set_message_gateway`, `set_message_endpoints`, `set_project_signers`, and `set_contract_relayes`, potentially redirecting messages, authorizing malicious remote senders, or blocking legitimate delivery.

Recommendation

Enforce owner initialization at framework level. The cleanest fix is for `#[default_impl]` to require a constructor shape such as `__constructor(env: Env, owner: Address, ...)` and automatically initialize ownership before invoking the user-defined constructor body. At minimum, deployments should fail closed unless the owner is set during construction.

Status

Resolved



[L-03] message-gateway-v4/src/lib.rs#process_gas_refund - Gas refunds are not verified or reserved per message

Description

The Stellar gateway treats gas reimbursement as relayer-supplied metadata rather than authenticated per-message state.

Vulnerability Details

The process request includes gas as an input field in `contracts/stellar/message-gateway-v4/src/interface.rs:74`, and `process()` forwards `request.gas` directly into the refund flow in `contracts/stellar/message-gateway-v4/src/lib.rs:301`.

However, the signed message material built for verification in `contracts/stellar/message-gateway-v4/src/lib.rs:313-345` does not bind either the relayer or the claimed gas amount.

`process_gas_refund` enforces only `max_gas_amount` in `contracts/stellar/gas_handler/src/lib.rs:119-125`, then checks the client contract's current allowance and balance in `contracts/stellar/gas_handler/src/lib.rs:135-141` before transferring funds in `contracts/stellar/gas_handler/src/lib.rs:147-154`.

Impact

An authorized relayer can claim any refund amount up to the configured cap for a processed message instead of only the true execution cost. One processed message can also consume shared refund capacity that another valid in-flight message needs. This creates capped overcharging, incentive misalignment, and liveness risk.

Recommendation

If the path is intended as true reimbursement, bind gas to signed message data or derive the reimbursable amount deterministically per `tx_id`, and reserve or escrow refund budget per message.

If the intended behavior is fixed or capped relayer compensation, document that clearly and remove assumptions that the paid amount corresponds to verified execution cost.

Status

Acknowledged

[L-04] message-gateway-v4/src/lib.rs#send - Direct account invocation is accepted on-chain but silently dropped off-chain

Description

The on-chain gateway and off-chain relayer enforce inconsistent call-topology rules for `gateway.send`, causing direct account invocations to succeed on-chain but be silently dropped off-chain.

Vulnerability Details

`MessageGatewayV4::send` in `contracts/stellar/message-gateway-v4/src/lib.rs:223-261` accepts any top-level invocation. It performs `request.sender.require_auth()` but does not check the caller's type, call stack, or `env.invoker()`.

When the gate passes, `send` charges the fee through `FeeHandler::process_fee`, increments `chain_details.next_tx_id`, and emits a `SendRequested` event.

The relayer parser at `src/drivers/StellarV4.ts:740-788` walks every `invokeHostFunction` operation. If it finds one whose target contract is the configured gateway address, it throws in `src/drivers/StellarV4.ts:776-778`:

```
if (calledContract.toLowerCase() === this.config.gatewayAddress?.toLowerCase()) {  
  throw new Error('Gateway can only be called by a contract, not directly by an  
  account');  
}
```

The throw is caught in the per-event try/catch in `src/drivers/StellarV4.ts:846-849`, logged at debug level, and the event is skipped.

Proof of Concept

```
#[test]  
fn test_account_direct_send_consumes_state() {
```

```

let env = Env::default();
env.mock_all_auths();
let owner = Address::generate(&env);
let sender = Address::generate(&env);
let gateway_id = env.register(MessageGatewayV4, (1u64, owner.clone(), 42u128));
let gateway = MessageGatewayV4Client::new(&env, &gateway_id);
let fh_id = env.register(FeeHandler, (&owner, &gateway_id));
let fh = fee_handler::FeeHandlerClient::new(&env, &fh_id);
fh.set_fee_amount(&0);
gateway.set_fee_handler(&fh_id);
gateway.set_system_enabled(&true);
let initial_tx_id = gateway.get_next_tx_id();
let request = SendRequest {
    sender: sender.clone(),
    recipient: Bytes::from_array(&env, &[1u8, 2, 3]),
    destination_chain: 2,
    chain_data: Bytes::from_array(&env, &[0xAA, 0xBB, 0xCC, 0xDD]),
    confirmations: 0,
};
let returned_tx_id = gateway.send(&request);
let chain_details_after = gateway.get_chain_details();
assert_eq!(returned_tx_id, initial_tx_id);
assert_eq!(chain_details_after.next_tx_id, initial_tx_id + 1);
assert!(env.events().all().iter()
    .any(|(c, _, _)| c == &gateway_id), "SendRequested event emitted");
}

```

Impact

A user who submits a top-level `gateway.send` transaction loses the fee, consumes a `tx_id`, and emits an event, but the relayer silently discards it. The cross-chain message is never delivered, and there is no on-chain feedback that the message was lost.

Recommendation

Either enforce the wrapper-contract requirement on-chain before charging fees or consuming `tx_id`, or relax the off-chain rejection if direct account submissions are intended to be supported. If direct calls are supported, the gateway event should emit

the authenticated sender explicitly instead of requiring the relay to infer it from transaction shape.

Status

Resolved

[L-05] gas_handler/src/lib.rs#process_gas_refund - Fee-on-transfer gas tokens can block inbound message delivery

Description

Function `process_gas_refund` pulls the gas fee from the client in one SEP-41 transfer and then forwards the same nominal amount to the relay in a second transfer. Fee-on-transfer tokens can cause the second transfer to fail.

Vulnerability Details

At `contracts/stellar/gas_handler/src/lib.rs:147-154`, the handler performs two independent SEP-41 operations using the same `amount`:

```
token_client.transfer_from(
    &gas_handler_address,
    &client_contract,
    &gas_handler_address,
    &amount,
);
token_client.transfer(&gas_handler_address, &relay, &amount);
```

For a fee-on-transfer token, the first transfer credits less than `amount` to the gas handler. The second transfer still attempts to send the full `amount`, which can exceed the gas handler's balance and panic.

The codebase does not warn against this configuration. The README states that the system supports any Stellar token for fee payment in `contracts/stellar/README.md:104-110`, while neither `gas_handler::set_token_contract` nor `fee_handler::set_token_contract` validates the token behavior.

Proof of Concept

```
#[test]
fn test_b_fot_token_traps_process_gas_refund() {
    let env = Env::default();
    env.mock_all_auths();
    let owner = Address::generate(&env);
    let gateway = Address::generate(&env);
    let client_contract = Address::generate(&env);
    let relayer = Address::generate(&env);
    let gas_amount: u64 = 100;
    let amount: i128 = gas_amount as i128;
    let (token_addr, token) = deploy_fot_token(&env, 500);
    let (gh_id, _gh_client) = deploy_gas_handler(
        &env, &owner, &gateway, &client_contract, &token_addr, gas_amount,
    );
    token.mint(&client_contract, &(amount * 2));
    token.approve(&client_contract, &gh_id, &amount, &9999);
    let result = std::panic::catch_unwind(AssertUnwindSafe(|| {
        let gh = gas_handler::GasHandlerClient::new(&env, &gh_id);
        gh.process_gas_refund(&client_contract, &relayer, &gas_amount);
    }));
    assert!(result.is_err(), "expected trap from process_gas_refund with FoT token");
    assert_eq!(token.balance(&relayer), 0);
}
```

Impact

Every inbound cross-chain message using the affected gas token can revert until the operator swaps the gas token. No funds are lost, but message delivery can be bricked.

Recommendation

Forward the actual credited delta instead of assuming conservation:

```
let balance_before = token_client.balance(&gas_handler_address);
token_client.transfer_from(
    &gas_handler_address,
    &client_contract,
```

```

    &gas_handler_address,
    &amount,
);
let delta = token_client.balance(&gas_handler_address) - balance_before;
token_client.transfer(&gas_handler_address, &relayer, &delta);

```

Also update the README to state that the gas token must be value-conserving and must not charge transfer fees. Optionally add a deploy-time probe in `set_token_contract` to verify exact transfer behavior.

Status

Resolved

[L-06] message-gateway-v4/src/storage.rs - Contract addresses can be configured as signers but can never satisfy signature checks

Description

The gateway signer setters accept any `Address`, including contract addresses, but runtime verification only supports ed25519 account signatures.

Vulnerability Details

The signer setters accept any address in:

- `contracts/stellar/message-gateway-v4/src/storage.rs:187-209`
- `contracts/stellar/message-gateway-v4/src/storage.rs:221-244`
- `contracts/stellar/message-gateway-v4/src/storage.rs:322-348`

At verification time, the gateway validates ed25519 signatures in `contracts/stellar/message-gateway-v4/src/lib.rs:356-360` and compares the recovered public key against bytes derived from configured signers in `contracts/stellar/message-gateway-v4/src/lib.rs:362-380` and `contracts/stellar/message-gateway-v4/src/lib.rs:750-767`.

For account addresses, the derived bytes correspond to an ed25519 public key. For contract addresses, they correspond to the contract hash, which has no associated ed25519 private key.

Impact

A contract address can be counted toward the chain, VIA, or project signer threshold during configuration but can never satisfy signature verification at runtime. If quorum is configured assuming the contract address can sign, message processing can become impossible.

Recommendation

Reject contract addresses at the signer-setter boundary and reserve signer roles for account addresses only. If contract-based authorization is intended, implement a separate verification path.

Status

Resolved

[L-07] `message-gateway-v4/src/storage.rs` - Duplicate signer entries can create an impossible quorum

Description

The signer-setting paths accept duplicate addresses because they only check `required` against the raw array length.

Vulnerability Details

The affected checks are located in:

- `contracts/stellar/message-gateway-v4/src/storage.rs:191-194`
- `contracts/stellar/message-gateway-v4/src/storage.rs:225-228`
- `contracts/stellar/message-gateway-v4/src/storage.rs:330-332`

A configuration such as `[A, A]` with `required = 2` is accepted during setup.

At processing time, signatures from the same public key are explicitly rejected as duplicates in `contracts/stellar/message-gateway-v4/src/lib.rs:350-351`, and each signer can only contribute once toward the threshold checked in `contracts/stellar/message-gateway-v4/src/lib.rs:386-395`.

Impact

The configured quorum can become unreachable even though the contract accepted the configuration. This can cause denial of service for message processing under the affected signer set.

Recommendation

Reject duplicate signer addresses when storing signer sets and validate `required` against the number of unique signers rather than the raw vector length.

Status

Resolved

[L-08] fee-handler/src/lib.rs#process_fee - Negative fee values are accepted and can break fee processing

Description

The fee handler does not reject negative default fee amounts or negative per-sender `max_fee` values.

Vulnerability Details

If `fee_amount < 0`, `process_fee()` still proceeds through the fee path, and the gateway `send()` flow still calls into the handler. This can push every sender using the default-fee path into a downstream token transfer with a negative amount.

Under the standard Soroban token model, negative token transfer amounts should revert rather than execute.

The same validation gap exists for per-sender `max_fee` values. If `max_fee < 0`, the cap check in `process_fee()` becomes unsatisfiable for any normal positive fee because `fee_amount > fee_max_cap` becomes true immediately. The handler then fails with `FeeAmountNotAuthorized`.

Impact

A negative default `fee_amount` can block fee-paying sends system-wide until governance or the owner corrects the configuration. A negative per-sender `max_fee` can deterministically deny service to the affected sender.

Recommendation

Validate fee configuration at write time and reject negative values for both the default fee amount and per-sender fee caps.

Status

Resolved

[L-09] message-gateway-v4/src/lib.rs#send - Risk of protocol archival due to Soroban instance TTL expiry

Description

The Stellar gateway, fee-handler, and gas-handler rely on Soroban instance storage, but none of them refresh the TTL of their instance entries.

Vulnerability Details

A repo-wide search shows no `instance().extend_ttl(...)` usage under `contracts/stellar/`. The only TTL bumps in scope target persistent entries, namely `ProcessedTransfers` in `contracts/stellar/message-gateway-v4/src/lib.rs:293-297` and `ChainDetails` in `contracts/stellar/message-gateway-v4/src/lib.rs:651-656`.

Gateway hot paths `send()` and `process()` read instance-backed configuration such as `is_system_enabled`, without refreshing instance TTL in `contracts/stellar/message-gateway-v4/src/lib.rs:223-311`.

The fee-handler stores and consumes fee configuration through instance storage in `contracts/stellar/fee-handler/src/lib.rs:132-320`, and the gas-handler does the same for refund configuration in `contracts/stellar/gas_handler/src/lib.rs:45-162`.

Impact

If the gateway instance archives, the core Stellar message flow stops. If a configured fee-handler or gas-handler archives, the routes that depend on that handler stop until restoration or reconfiguration.

Recommendation

Add a helper around `env.storage().instance().extend_ttl(...)` in each Stellar contract using instance storage, and call it at the start of every public entrypoint that reads or writes instance-backed state.

Status

Acknowledged

[L-10] message-client/src/abi.rs#abi_decode - Extra ABI fields are silently ignored

Description

The `abi_decode()` accepts an arbitrary `types` slice but only decodes and returns the first three positions. Additional ABI fields are dropped without an explicit error.

Vulnerability Details

The decoder API suggests generic support for a caller-provided set of Solidity ABI types, but the implementation stops after decoding indices 0, 1, and 2.

```
pub fn abi_decode(
    env: &Env,
    encoded: &Bytes,
    types: &[AbiType],
) -> (Option<AbiValue>, Option<AbiValue>, Option<AbiValue>) {
    let binding = encoded.to_buffer:::<65536>();
    let data = binding.as_slice();
    let mut offset = 0usize;
    let mut result = (None, None, None);
    for (idx, abi_type) in types.iter().enumerate() {
        if idx >= 3 {
            break; // Only support up to 3 return values for now
        }
    }
}
```

Callers supplying more fields do not receive a rejection or partial-support notice. Instead, later values are silently ignored. This creates a mismatch between the apparent interface contract and the actual decoding behavior.

Impact

Security-relevant data placed beyond the third field can be omitted from application logic without any visible failure. That makes protocol evolution harder and increases the risk that future payload formats are integrated incorrectly.

Recommendation

Either reject any `types` array longer than the supported output tuple or redesign the decoder to return all requested fields. Silent truncation should be replaced with explicit, typed failure.

Status

Resolved

[L-11] message-client/src/abi.rs#abi_decode - Malformed payloads can panic

Description

The ABI decoder performs multiple fixed-width and dynamic slices without sufficient bounds checks. Crafted short payloads can therefore abort execution instead of failing cleanly.

Vulnerability Details

The `abi_decode()` reads static words and later follows dynamic offsets into the payload using direct slice operations and `unwrap()` conversions.

```
AbiType::Bytes => {  
    // Read offset pointer  
    let offset_bytes: [u8; 4] = data[offset + 28..offset +  
32].try_into().unwrap();  
    let data_offset = u32::from_be_bytes(offset_bytes) as usize;  
    // Read length at the offset  
    let len_bytes: [u8; 4] = data[data_offset + 28..data_offset +  
32].try_into().unwrap();  
    let len = u32::from_be_bytes(len_bytes) as usize;  
    // Read the bytes data  
    let bytes_data = &data[data_offset + 32..data_offset + 32 + len];  
    let bytes = Bytes::from_slice(env, bytes_data);  
}
```

When the input is shorter than expected or the offsets point outside the available range, these operations panic. Because the decoder sits on a cross-chain message boundary, malformed input from another domain can be turned into a deterministic trap on the destination side.

Impact

Malformed payloads can cause message processing to revert rather than returning a structured decode failure. This is primarily a denial-of-service and robustness problem, but it affects a path that may receive attacker influenced bytes from external chains.

Recommendation

Add explicit length and offset validation before every slice or conversion, and return a typed decode error instead of panicking. The decoder should treat invalid payloads as bad input, not as unrecoverable internal state corruption.

Status

Acknowledged

[L-12] message-client/src/abi.rs#abi_decode - Dynamic ABI words use only the low 4 bytes

Description

Dynamic offsets and lengths are narrowed from the last 4 bytes of a 32-byte ABI word. This accepts non-canonical encodings and makes distinct ABI words decode identically.

Vulnerability Details

The decoder reads only the trailing 4 bytes of a 32-byte ABI word for both offsets and lengths.

```
let offset_bytes: [u8; 4] = data[offset + 28..offset + 32].try_into().unwrap();
let data_offset = u32::from_be_bytes(offset_bytes) as usize;
// Read length at the offset
let len_bytes: [u8; 4] = data[data_offset + 28..data_offset +
32].try_into().unwrap();
let len = u32::from_be_bytes(len_bytes) as usize;
```

Solidity ABI uses full 32-byte big-endian integers, so the current implementation treats multiple different words as the same offset or length as long as their low 4 bytes match. Even when typical payload sizes stay small, this is still ABI non-compliance and an unnecessary source of ambiguity.

Impact

Non-canonical or maliciously encoded payloads can decode successfully into values that were not actually uniquely committed by the upstream encoder. This weakens parser strictness at a cross-chain trust boundary and makes fuzzing or compatibility behavior harder to reason about.

Recommendation

Parse the full 32-byte word and assert that the upper bytes are zero before narrowing to a smaller integer type. Reject any dynamic offset or length that is not canonically encoded or exceeds local bounds.

Status

Acknowledged

[L-13] fee-handler/src/lib.rs#set_max_fee - Sender max-fee protection requires dual authorization

Description

The `set_max_fee()` is guarded by both `#[only_owner]` and `sender.require_auth()`, even though the interface describes the cap as sender-controlled. The function is not dead code, but its authorization model is inconsistent with the stated feature.

Vulnerability Details

The handler documentation says the maximum fee is "set by the sender themselves," yet the implementation requires both owner authorization and sender authorization in the same call.

```
#[only_owner]
fn set_max_fee(env: &Env, sender: Address, amount: i128) -> Result<(), Error> {
    sender.require_auth();
    env
        .storage()
        .instance()
        .set(&DataKey::MaxFee(sender.clone()), &amount);
}
```

Soroban can satisfy multi-address auth for some flows, so the function is not universally uncallable, but the design still makes self-service fee caps awkward and potentially unusable for contract senders or for integrations that assumed unilateral sender control.

Impact

Users may believe they can independently enforce a fee ceiling when in practice they still depend on owner cooperation or more complex multi-auth transaction construction. This weakens the operational value of the fee-cap feature and can leave approvals exposed longer than users expect.

Recommendation

Remove `#[only_owner]` from `set_max_fee()` and let the sender authorize their own cap directly. If operator oversight is required, expose a separate owner-controlled safety feature rather than overloading a user-protection mechanism with dual authorization.

Status

Resolved

[L-14] fee-handler/src/lib.rs#__constructor - Wrong storage key initialized for source fee

Description

The fee-handler constructor stores `SOURCE_FEE_AMOUNT` in `DataKey::FeeAmount` instead of `DataKey::SourceFeeAmount`. This inverts the intended defaults immediately after deployment.

Vulnerability Details

During initialization, the constructor sets the owner and gateway and then writes the constant `SOURCE_FEE_AMOUNT` into the primary fee slot.

```
fn __constructor(env: &Env, owner: Address, gateway: Address) -> Result<(), Error> {  
    set_owner(env, &owner);  
    set_admin(env, &gateway);  
    env  
        .storage()  
        .instance()  
        .set(&DataKey::FeeAmount, &SOURCE_FEE_AMOUNT);  
}
```

The dedicated source-fee key is left unset. As a result, the contract boots with a nonzero token-fee amount even before the token-fee configuration is fully in place, while the source-fee path does not receive the default the constant name implies.

Impact

Fresh deployments can start from an unexpected charging configuration and fail in confusing ways if operators assume the constructor populated the source-fee state correctly. This increases the chance of broken launches and mispriced fee collection.

Recommendation

Initialize `DataKey::SourceFeeAmount` with `SOURCE_FEE_AMOUNT` and leave `DataKey::FeeAmount` at its intended default unless explicitly configured.

Status

Resolved

[L-15] fee-handler/src/lib.rs#process_fee - Source-fee path mixes allowance checks with direct transfer

Description

The source-fee branch checks allowance from the sender to the fee-handler, but it then executes a direct `transfer()` rather than an allowance-consuming `transfer_from()`. The auth and accounting assumptions of the two paths do not match.

Vulnerability Details

The `check_balances()` validates that the sender has enough native token balance and that the allowance granted to the fee-handler is at least `source_fee_amount`.

```
let allowance = token_client.allowance(from, spender);
if allowance < amount {
    panic_with_error!(env, Error::FeeAmountNotAuthorized);
}
[...]
```

```
if is_source_fee && source_fee_amount > 0 {
    let native_address = Address::from_str(env, NATIVE_TOKEN_ADDRESS);
    let native_client = TokenClient::new(env, &native_address);
    Self::check_balances(
        env,
        &native_client,
        &sender,
        &fee_handler_address,
        source_fee_amount,
    );
    native_client.transfer(&sender, &recipient, &source_fee_amount);
}
```

This means the branch checks for one authorization model but spends using another, relying on propagated auth context rather than on the allowance it just validated.

Impact

Source-fee collection can fail unexpectedly even when preflight checks appear to approve it, and the code becomes harder to audit because the actual spend path does not match the authorization model suggested by the checks. Integrators can misconfigure approvals and still see runtime failures.

Recommendation

Make the source-fee spend path consistent with the pre-checks by using `transfer_from()`, or redesign the flow so direct sender auth is the only requirement and remove the allowance checks. The implementation should enforce one coherent payment model.

Status

Resolved

[L-16] message-gateway-v4/src/lib.rs#process - Signer rotation can invalidate in-flight signature sets

Description

The Stellar gateway verifies inbound messages against signer sets and required counts that are live when `process()` runs, not against the configuration that was active when the signature set was collected.

Vulnerability Details

Global VIA signers are replaced through `contracts/stellar/message-gateway-v4/src/lib.rs:198-202` and stored in `contracts/stellar/message-gateway-v4/src/storage.rs:187-209`.

Global chain signers are replaced through `contracts/stellar/message-gateway-v4/src/lib.rs:205-213` and stored in `contracts/stellar/message-gateway-v4/src/storage.rs:221-243`.

Project signers are replaced through `contracts/stellar/message-gateway-v4/src/lib.rs:445-452` and stored in `contracts/stellar/message-gateway-v4/src/storage.rs:322-347`.

During validation, the gateway always loads the current chain, VIA, and project signer sets in `contracts/stellar/message-gateway-v4/src/lib.rs:342-345` and enforces live membership and live thresholds in `contracts/stellar/message-gateway-v4/src/lib.rs:347-397`.

There is no signer-set epoch, activation timestamp, grace period, or overlap logic in the signed message or verification path.

Impact

A signature set collected under signer set A may stop being valid after operators rotate to signer set B or raise a threshold. This can cause per-message liveness failures for in-flight traffic and may require collecting a fresh bundle or using a manual recovery path.

Recommendation

Bind approvals to a signer-set version or activation epoch, or accept both old and new signer configurations for a defined overlap window during rotation. If immediate cutover semantics are intentional, document clearly that signer updates take effect immediately and can invalidate already-collected bundles.

Status

Acknowledged

[L-17] message-gateway-v4/src/lib.rs#is_zero_address - Zero-address handler disable logic is broken

Description

The gateway attempts to use a zero-address sentinel to disable handlers, but the helper compares raw zero bytes against XDR-encoded addresses. As written, the disable check never matches real address representations.

Vulnerability Details

The `is_zero_address()` builds a `Bytes` value containing 32 zero bytes and compares it to the argument passed by callers.

```
fn is_zero_address(env: &Env, address: Bytes) -> bool {
    let zero_bytes = Bytes::from_array(env, &ZERO_BYTES);
    zero_bytes == address
}
[...]
```

```
if let Some(gas_handler) = gas_handler {
    let is_address_valid = !Self::is_zero_address(env,
gas_handler.clone().to_xdr(env));
    if is_address_valid {
```

This means branches that are supposed to skip handler configuration or treat a handler as disabled do not behave as designed, and operators cannot rely on a zero sentinel to turn optional components off.

Impact

Handler disablement becomes unreliable and operationally surprising. Once a POS, gas, or fee handler is configured, attempts to disable or bypass it using the documented zero-address convention can silently fail, which can complicate incident response or maintenance.

Recommendation

Replace the zero-sentinel pattern with explicit optional storage semantics such as `Option<Address>`, or compare a normalized raw-address representation rather than full

XDR. The API should make handler removal an explicit supported operation instead of relying on a representation-sensitive sentinel.

Status

Acknowledged



[L-18] storage.rs#ProcessedTransfers - Replay protection keyed only by tx_id

Description

Replay protection is stored as `ProcessedTransfers(tx_id)` with no `source_chain_id` namespace. This makes uniqueness depend entirely on correct global `tx_id` prefix management rather than on both source domain and message identifier.

Vulnerability Details

The gateway checks and sets replay state using `DataKey::ProcessedTransfers(request.tx_id)` in `process()`.

```
/// Storage key for tracking processed transfers by transaction ID
ProcessedTransfers(u128),
```

```
if env
    .storage()
    .persistent()
    .get(&DataKey::ProcessedTransfers(request.tx_id))
    .unwrap_or(false)
{
    return Err(Error::TransactionAlreadyProcessed);
}
[..]
env
    .storage()
    .persistent()
    .set(&DataKey::ProcessedTransfers(request.tx_id), &true);
```

Since `source_chain_id` is omitted from the replay key, two messages from different sources that collide on `tx_id` cannot be distinguished by storage. The design assumes that `message_prefix * 10^23 + sequence` will remain globally unique across all source domains and deployments, but that assumption becomes fragile under misconfiguration, multiple independent gateway deployments, or future topology changes.

Impact

Source-domain collisions can cause false duplicate rejection or enable replay behavior across unrelated message domains. Even if the current deployment keeps prefixes unique, the storage scheme bakes in a cross-domain coupling that reduces safety margins and makes future multi-source operation harder to reason about.

Recommendation

Namespace replay protection by both source and transaction identifier, for example by storing `ProcessedTransfers(source_chain_id, tx_id)`. This should be paired with explicit destination-chain enforcement so replay separation is complete across both source and destination domains.

Status

Acknowledged

[L-19] StellarV4 driver & stellar-utils - Testnet network is hardcoded or silently defaulted across four sites

Description

Four sites in the Stellar driver and stellar-utils layer silently default to or hardcode the Stellar testnet passphrase. A mainnet operator who omits or forgets network configuration silently builds testnet-passphrase transactions in any of these paths.

Vulnerability Details

The first site is at `src/drivers/StellarV4.ts:74`, which applies a module-init fallback `const STELLAR_NETWORK = process.env.STELLAR_NETWORK ?? 'testnet'`. When the environment variable is unset, the driver silently selects testnet. The second site is `simulateTransaction` at `src/utls/stellar-utils.ts:247`, which calls `.setNetworkPassphrase(Networks.TESTNET)` with no override parameter exposed to callers. The third site is `setTokenTrustLineFlags` at `src/utls/stellar-utils.ts:195`, which hardcodes the testnet passphrase inside the `TransactionBuilder` constructor and exposes no `networkPassphrase` parameter on the function signature. The fourth site is `submitStellarTransaction` at `src/utls/stellar-utils.ts:162`, which uses `networkPassphrase = Networks.TESTNET` as the default parameter value.

None of these paths require an explicit network parameter from the operator, and none fail closed when the configuration is missing.

Impact

Stellar's submission layer rejects mismatched-passphrase transactions, so the failure is loud at submit time and the operator receives immediate feedback. There is no silent corruption, and no cross-chain hash binding depends on the Stellar passphrase. The consequence is operational availability and deployment safety for misconfigured mainnet environments, not direct funds at risk.

Recommendation

Remove all four defaults and require explicit passphrase configuration. At module init, reject any `STELLAR_NETWORK` value other than `mainnet` or `testnet`. Add `networkPassphrase`

as a required parameter to `simulateTransaction` and `setTokenTrustLineFlags`. Remove the default value from `submitStellarTransaction` and reject calls where the passphrase is absent.

Status

Acknowledged



QA

[Q-01] Message provenance relies on brittle conventions instead of explicit commitments

Description

Several message-authenticity checks in the Stellar flow depend on conventions, inferred context, or permissive validation rather than a single explicit provenance model.

Vulnerability Details

`encode_via_message` omits the gateway contract address from the signed pre-image in `contracts/stellar/message-gateway-v4/src/lib.rs:698-736`, and the off-chain mirror in `src/utils/stellar-encoding.ts:57-85` does the same.

The Stellar driver reconstructs the sender from transaction shape because the gateway event omits it in `contracts/stellar/message-gateway-v4/src/events.rs:8-15`, and `src/drivers/StellarV4.ts:740-780` infers sender identity from the top-level call.

`validate_chain_sender` in `contracts/stellar/message-client/src/lib.rs:403-416` can pass vacuously if both the configured endpoint and incoming sender are empty, because `get_endpoint_by_chain_id` falls back to empty bytes in `contracts/stellar/message-client/src/lib.rs:395-401`.

The gateway verifies each `ed25519` signature in `contracts/stellar/message-gateway-v4/src/lib.rs:356-360`, checks configured signer-set membership in `contracts/stellar/message-gateway-v4/src/lib.rs:375-380`, and still pushes otherwise valid but unlisted signatures into `valid_signatures` in `contracts/stellar/message-gateway-v4/src/lib.rs:383`.

Impact

None of the individual gaps currently produces a clean standalone exploit under the default VIA flow, but together they show that security-critical message properties are only indirectly enforced. This can cause wasted gas, brittle sender provenance, permissive validation boundaries, and weak defense-in-depth.

Recommendation

Bind the signed message to the current gateway address, emit the authenticated sender directly in the Stellar gateway event, and fail closed on malformed provenance inputs by rejecting missing endpoints, empty senders, and signatures from keys that are not members of any configured signer set.

Status

Acknowledged

[Q-02] ABI decoding relies on permissive internal assumptions instead of explicit validation

Description

The Stellar ABI helper in `contracts/stellar/message-client/src/abi.rs` relies on permissive internal assumptions where unsupported or malformed inputs should fail explicitly.

Vulnerability Details

Narrow scalar decoding does not enforce canonical ABI words in `contracts/stellar/message-client/src/abi.rs:269-330`. Unsigned integers are narrowed without requiring zero high bytes, signed integers are narrowed without validating full sign extension, and booleans are derived from the low byte without requiring the word to be exactly 0 or 1.

Dynamic Bytes and String decoding in `contracts/stellar/message-client/src/abi.rs:332-365` does not validate offsets and lengths before slicing.

The documented three-value limit of `abi_decode` is visible from the fixed return shape in `contracts/stellar/message-client/src/abi.rs:247-266`, but it is not enforced as an explicit precondition. If a caller supplies `types.len() > 3`, the helper silently stops decoding after the third value.

Impact

This does not appear to create a direct external exploit under the current threat model because `abi_decode` is an internal helper, but it weakens robustness and can make future integrations harder to reason about.

Recommendation

Enforce canonical ABI words before narrowing scalar types, validate offsets and lengths before decoding Bytes and String, return controlled decode errors instead of relying on unchecked slicing, and add an explicit precondition such as `assert!(types.len() <= 3)`.

Status

Acknowledged

[Q-03] Several configuration paths rely on brittle sentinels or undocumented numeric assumptions

Description

Some Stellar-side configuration and state handling depends on implicit sentinel values or undocumented numeric limits rather than explicit types and checked invariants.

Vulnerability Details

The gateway uses `is_zero_address` in `contracts/stellar/message-gateway-v4/src/lib.rs:745-747` to decide whether to persist POS, gas, and fee handler addresses during initialization in `contracts/stellar/message-gateway-v4/src/lib.rs:183-192`.

The initializer request in `contracts/stellar/message-gateway-v4/src/interface.rs:116-121` models these handler fields as mandatory Address values rather than optional ones.

The gateway derives its initial transaction counter as `message_prefix * TX_ID_FACTOR` in `contracts/stellar/message-gateway-v4/src/lib.rs:109-112`, then increments `next_tx_id` on every `send()` in `contracts/stellar/message-gateway-v4/src/lib.rs:243-246`, but neither step uses checked arithmetic or validates the maximum safe `message_prefix`.

The fee handler uses a hardcoded magic sentinel `FREE_FEE_AMOUNT = 9999` in `contracts/stellar/fee-handler/src/lib.rs:36` and interprets that exact stored whitelist amount as a free pass in `contracts/stellar/fee-handler/src/lib.rs:81-93`.

Impact

These issues create configuration brittleness and maintainability risk. The strongest immediate effect is accidental misbehavior under edge-case operator choices rather than a concrete externally triggerable exploit.

Recommendation

Replace sentinels and implicit numeric assumptions with explicit types and checked invariants. Use `Option<Address>` for optional handlers, validate `message_prefix`, use checked arithmetic for `tx_id`, and replace `FREE_FEE_AMOUNT = 9999` with an explicit representation such as `Option<i128>` or an enum distinguishing `Free` from `CustomFee(i128)`.

Status

Acknowledged

[Q-04] Reference client silently truncates inbound dust during denormalization

Description

The reference Stellar client converts canonical 18-decimal inbound amounts to its local 7-decimal token by integer division.

Vulnerability Details

The affected logic is located in `contracts/stellar/client-test-v4/src/lib.rs:86`:

```
let local_amount = normalized_amount / 10i128.pow(11); // 18 - 7 = 11
```

This truncates any remainder below 10^{11} canonical base units.

Impact

This causes bounded dust loss on each inbound transfer processed by the example client. For ordinary transfer sizes, the lost amount is negligible, but repeated tiny transfers or round-trips can accumulate dust. Integrators may copy this example and inherit both the truncation behavior and the hardcoded `18 -> 7` decimal assumption.

Recommendation

Make decimal conversion explicit and configuration-driven in the example client. Document that integrators must align the divisor with the actual source and destination token decimals instead of copying the constant blindly.

Status

Acknowledged

[Q-05] Interface documentation does not match the generated client API or runtime behavior

Description

The `contracts/stellar/message-gateway-v4/src/interface.rs:142-202` documentation does not accurately describe how some parts of the Stellar gateway are actually used.

Vulnerability Details

The client example imports `MessageGatewayV4Client` from an invalid path and calls `gateway.send(&send_request)?`, even though the generated client API takes an owned `SendRequest` and `send()` returns `u128` rather than `Result`.

The constructor docs still refer to `deployer`, while the actual argument is `owner`.

Some method comments do not match runtime semantics. For example, the `process` docs describe `SystemNotEnabled` as an error return, but the implementation panics on that path in `contracts/stellar/message-gateway-v4/src/lib.rs:263`. The `send` docs also say the function rejects a zero recipient, while the implementation checks whether the destination `Bytes` value is empty in `contracts/stellar/message-gateway-v4/src/lib.rs:227`.

Impact

Integrators and maintainers may form the wrong mental model of the API or error behavior from outdated comments and examples.

Recommendation

Update the interface comments and examples in `contracts/stellar/message-gateway-v4/src/interface.rs:142-202` so they match the generated client API and actual runtime semantics.

Status

Acknowledged

[Q-06] Unused definitions leave stale surface area in the Stellar codebase

Description

Several Stellar contracts contain unused traits, constants, events, and error variants that no longer participate in the live implementation.

Vulnerability Details

Examples include:

- `MessageProcess` in `contracts/stellar/message-client/src/lib.rs:440`
- Unused storage constants such as `CHAIN_DETAILS_KEY` and `TEMPORARY_STORAGE_MIN_TTL` in `contracts/stellar/message-gateway-v4/src/storage.rs:15-42`
- Unused gateway events such as `SetOwner` and `SetTokenContract` in `contracts/stellar/message-gateway-v4/src/events.rs:20-122`
- Orphaned error variants in `contracts/stellar/message-gateway-v4/src/errors.rs:50`, `contracts/stellar/gas_handler/src/errors.rs:7-9`, and `contracts/stellar/fee-handler/src/errors.rs:7-14`

Impact

Stale definitions expand the apparent API surface, make audits slower, and increase the chance that future contributors rely on dead code paths or outdated intent.

Recommendation

Remove unused definitions or reconnect them to live code paths where they are still intended. Keeping the Stellar contracts leaner will make the real behavior easier to audit and reduce ambiguity for future maintenance.

Status

Acknowledged

[Q-07] fee-handler/src/lib.rs#process_fee - Source fee bypasses sender max-fee caps

Description

The sender's configured `max_fee` is enforced only for the primary token-fee branch, not for the source-fee branch. Owner controlled source fee routing can therefore bypass the sender's fee ceiling.

Vulnerability Details

The `process_fee()` calculates `fee_max_cap` and checks it only when evaluating the configured token `fee_amount`.

```
let fee_amount = Self::get_amount(env, sender.clone());
let is_source_fee = Self::get_source_fee_whitelist(env, sender.clone());
let source_fee_amount = Self::get_source_fee_amount(env);
let recipient = Self::get_fee_recipient(env);
if is_source_fee && source_fee_amount > 0 {
    let native_address = Address::from_str(env, NATIVE_TOKEN_ADDRESS);
    let native_client = TokenClient::new(env, &native_address);
    Self::check_balances(
        env,
        &native_client,
        &sender,
        &fee_handler_address,
        source_fee_amount,
    );
}
```

```

    native_client.transfer(&sender, &recipient, &source_fee_amount);
}
if fee_amount == 0 {
    return true;
}
let fee_max_cap = Self::get_max_fee(env, sender.clone());
let is_fee_capped = fee_max_cap != 0 && fee_amount > fee_max_cap;
if is_fee_capped {
    panic_with_error!(env, Error::FeeAmountNotAuthorized)
}

```

The source-fee logic is handled earlier and independently based on `source_fee_whitelist` and `SourceFeeAmount`, with no equivalent cap comparison. This makes the sender's max-fee protection incomplete even if `set_max_fee()` is fixed or successfully used.

Impact

Users who believe they capped their fee exposure can still be charged uncapped source fees whenever the owner enables that path for their address. The protection is therefore partial and can be bypassed through configuration rather than through code execution tricks.

Recommendation

Apply the sender-configured cap to total fees collected across all fee branches, or provide separate sender-controlled caps for each fee type. The enforcement point should cover whichever fee path the owner can enable.

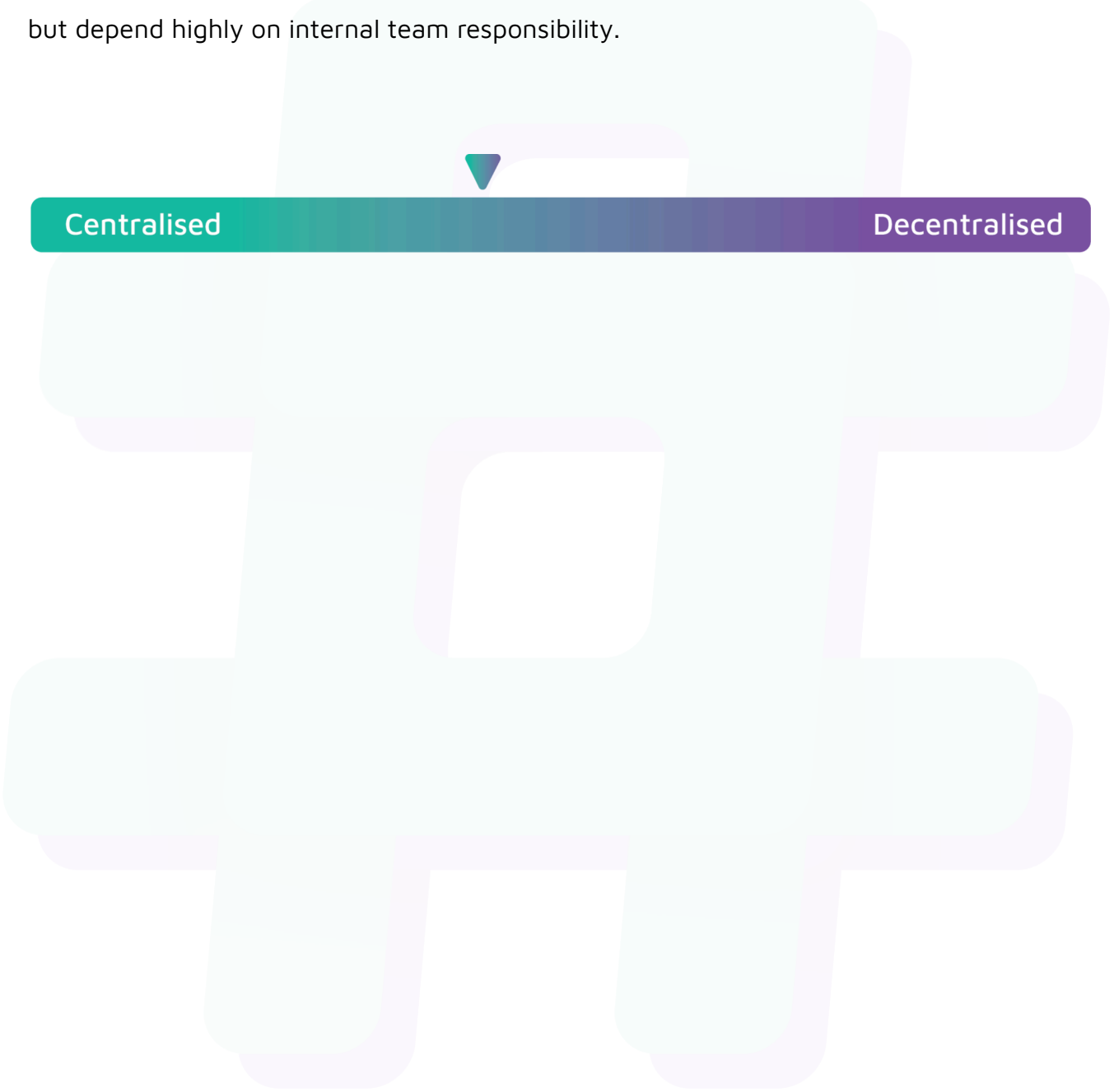
Status

Acknowledged

Centralisation

The Via Labs project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Via Labs project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged . Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.